

minishell

Fiches de soutenance

42 Lausanne · aalvard & dbomfim- · septembre 2026

A quoi sert ce document

Il ne remplace pas votre code, il vous rend capables de le défendre. Un correcteur de 42 ne lit presque jamais tout le depot : il tape des commandes, il regarde ce qui sort, et il pose des questions. Ce document suit exactement cette logique.

Partie 1 : la grille d'évaluation officielle, section par section. Pour chaque section : ce que le correcteur tape, ce que votre shell répond, ou c'est géré dans votre code, et le piège.

Partie 2 : le code module par module, avec les extraits qui comptent et le *pourquoi* de chaque choix. C'est la partie à lire en croise : Axel revise l'exécuteur de Davi, Davi revise le lexer d'Axel.

Partie 3 : la soutenance : qui a écrit quoi, les questions pièges avec leurs réponses, le résultat de la campagne adversariale du 23 septembre, et la checklist du jour J.

Les trois choses qui mettent zero, avant tout le reste

1. Un segfault. La grille dit mot pour mot : *for the duration of the defense, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0*. Un seul crash, n'importe quand, et c'est fini.

2. Une fuite mémoire. Le correcteur a le droit de sortir valgrind. Vos leaks à vous doivent être à zero.

3. Une erreur de norme, ou plus d'une variable globale. Flag, fin de l'évaluation.

Tout le reste se discute. Ces trois-là, non.

Partie 1

La grille d'evaluation, section par section

Source : la fiche officielle de 42evalhub pour le projet minishell, dans sa version de septembre 2026. Les 21 sections ci-dessous sont celles que le correcteur doit cocher. Lisez-les comme un script : c'est littéralement ce qu'il va faire.

1. Compile

```
make -n | grep -- -Wall
make re
make # ne doit rien recompiler
```

Ce que votre shell répond

Les trois flags **-Wall -Wextra -Werror** sont dans le Makefile. **make** relance juste apres un **make re** affiche *Nothing to be done for 'all'* : pas de relink.

Ou c'est gere, auteur : Axel

Makefile. La regle **%o: %c** compare les dates, donc aucun objet n'est reconstruit inutilement. Le bloc **ifeq (\$(shell uname), Darwin)** ajoute les chemins de la readline de Homebrew : macOS ne fournit que libedit, dont l'en-tete ne declare pas **rl_replace_line**.

Le piege

Un **make** qui relink est un flag. Verifiez-le avant l'eval, pas pendant.

Ce que tu reponds

On a un job GitHub Actions qui rejoue make re, la norminette et les 16 suites de tests a chaque push. Le badge du README est vert.

2. Simple Command & global variables

```
minishell$ /bin/ls
minishell$ ls
minishell$ # ligne vide, Entree
minishell$ \t # espaces et tabulations seules
```

Ce que votre shell répond

Les commandes s'exécutent. Une ligne vide ou blanche ne produit rien et laisse **\$?** inchangé. Il y a **exactement une variable globale** : **int g_signal**, dans **src/signals/signals.c**.

Ou c'est gere, auteur : Davi (main, signals) : Axel (run_line)

src/main.c pour la boucle, **src/utlis/ut_init_data.c** pour **run_line**, **src/signals/signals.c** pour la globale. Le lexer sur une ligne vide renvoie une liste NULL et **run_line** sort immédiatement.

Le piege

La question *combien de globales et pourquoi* est dans la grille. Il faut une reponse precise, pas *une seule, comme demande*.

Ce que tu reponds

Une seule : un int qui recoit le numero du signal. Un handler de signal ne peut pas recevoir de parametre ni rendre de valeur : il ne peut ecrire que dans une variable de portee globale. On y met uniquement le numero du signal, jamais un pointeur ni une donnee du shell, ce que le sujet interdit explicitement. Le code de sortie 130 est ensuite calcule a partir de cette variable dans main.

3. Arguments

```
minishell$ /bin/ls -la
minishell$ /usr/bin/wc -l Makefile
minishell$ grep include includes/minishell.h
```

Ce que votre shell repond

Les arguments sont decoupees sur les espaces **et les tabulations**, et transmis tels quels a **execve** via **cmd->argv**.

Ou c'est gere, auteur : Axel (lexer, parser) : Davi (execve)

src/lexer/lex_tokenize.c decoupe, **src/parser/parse_word.c** construit le tableau argv, **src/executor/exec_child.c** appelle execve.

Ce que tu reponds

Le lexer traite l'espace et la tabulation comme des separateurs. Davi a ajoute la tabulation le 22 septembre, c'etait un oubli trouve en testant.

4. echo

```
minishell$ echo
minishell$ echo bonjour
minishell$ echo -n sans retour
minishell$ echo -nnnn aussi
minishell$ echo -n -n encore
minishell$ echo -N pas une option
minishell$ echo -xn pas une option non plus
```

Ce que votre shell repond

-n et ses repetitions sont consommees tant que l'argument ne contient que des **n** apres le tiret. **-N**, **-xn**, **-n-** et le tiret seul sont affiches comme du texte, comme dans bash.

Ou c'est gere, auteur : Axel

src/builtins/bi_echo.c. La boucle avance tant que l'argument commence par - et n'est suivi que de **n**.

Ce que tu reponds

60 assertions dans **test_builtins** comparent notre echo a celui de bash, argument par argument.

5. exit

```
minishell$ exit 42 # relancer ensuite
minishell$ exit abc
minishell$ exit 1 2
minishell$ exit 256
minishell$ exit -1
```

Ce que votre shell repond

exit 42 sort avec 42. **exit abc** affiche *numeric argument required* et sort avec 2. **exit 1 2** affiche *too many arguments*, **ne sort pas** et rend 1. **exit 256** sort avec 0, **exit -1** avec 255 : le code est pris modulo 256.

Ou c'est gere, auteur : Axel

src/builtins/bi_exit.c. Le drapeau **shell->want_exit** fait sortir la boucle de **main**, ce qui permet de liberer proprement avant de rendre la main.

Le piege

Le bash 3.2 d'Apple ne gere pas **exit abc def** comme bash 5 : il regarde d'abord le nombre d'arguments, bash 5 regarde d'abord la validite du premier. Notre shell suit bash 5, qui est ce qui tourne sur les machines de l'ecole.

Ce que tu reponds

On ne fait jamais exit() depuis un builtin sauf exit lui-meme : on pose un drapeau et on sort de la boucle, sinon on saute le free final et on leake tout.

6. Return value of a process

```
minishell$ /bin/ls ; echo $?
minishell$ /bin/ls filequinexistepas ; echo $?
minishell$ commandebidon ; echo $?
minishell$ expr $? + $?
minishell$ /etc ; echo $?
```

Ce que votre shell repond

0 en cas de succes, 2 pour ls sur un fichier absent, **127** pour une commande introuvable, **126** pour un repertoire ou un fichier sans droit d'execution, **128 + N** quand le process meurt d'un signal. **expr \$? + \$?** fonctionne.

Ou c'est gere, auteur : Davi (wait, codes) : Axel (\$? dans l'expander)

src/executor/exec_wait.c : **WEXITSTATUS** si le fils est sorti normalement, **128 + WTERMSIG** s'il a ete tue. **src/executor/exec_single.c** pour 126 et 127 avant le fork. **\$?** est lu depuis **shell->last_exit** par l'expander.

Ce que tu reponds

last_exit vit dans la structure t_shell, pas dans une globale. C'est justement pour ca qu'on n'a besoin que d'une seule globale.

7. Signals

```
minishell$ ctrl-C # ligne vide
minishell$ ctrl-backslash # ne doit rien faire
minishell$ ctrl-D # quitte, RELANCER
minishell$ du texte ctrl-C # puis Entree : rien ne doit s'executer
minishell$ cat ctrl-C # puis echo $? -> 130
minishell$ cat ctrl-backslash # Quit (core dumped), $? -> 131
minishell$ cat ctrl-D # cat se termine
```

Ce que votre shell repond

Tous ces cas sont conformes, verifiez le 23 septembre dans un vrai terminal sur Mac et pilotes par pty sous Linux. **ctrl-C** pendant un heredoc annule le heredoc et rend le prompt.

Ou c'est gere, auteur : Davi

src/signals/signals.c : trois jeux de handlers. Au prompt, SIGINT ecrit un retour a la ligne puis **rl_on_new_line**, **rl_replace_line("", 0)** et **rl_redisplay** : c'est ce trio qui vide le buffer de readline. Pendant l'execution, SIGINT et SIGQUIT sont ignores par le pere pour que seul le fils meure. **src/signals/signals_heredoc.c** pose un handler sans **SA_RESTART**, ce qui fait remonter **EINTR** du **read** bloquant et annule le heredoc.

Le piege

Le point le plus teste de toute la grille : *the buffer should be clean too, press Enter to make sure nothing from the previous line is executed*. Sans **rl_replace_line**, la ligne tapee reste dans readline et part a l'execution au prochain Entree.

Ce que tu reponds

rl_catch_signals = 0 dans *main* : on desactive la gestion interne de readline pour poser la notre. Le 130, c'est 128 + SIGINT, calcule dans *check_prompt_signal* a partir de la globale.

8. Double Quotes

```
minishell$ echo "cat lol.c | cat > lol.c"
minishell$ echo "un deux"
minishell$ echo "a"b"c"
```

Ce que votre shell repond

Le contenu entre guillemets est traite comme un seul mot. Les pipes et les chevrons a l'interieur ne sont jamais interpretes. Les espaces multiples sont conservees.

Ou c'est gere, auteur : Axel

src/lexer/lex_quotes.c : un automate a trois etats (NORMAL, SQUOTE, DQUOTE). Tant qu'on est dans un etat quote, | et > sont accumules comme des caracteres ordinaires, pas reconnus comme operateurs.

Ce que tu reponds

Les quotes sont conservees dans le token jusqu'a l'expander. Si on les enlevait dans le lexer, on ne saurait plus, au moment d'expanser, si un dollar etait entre simples quotes ou non.

9. Single Quotes

```
minishell$ echo '$USER'
minishell$ echo ''
minishell$ echo 'a | b > c'
minishell$ echo ' espaces '
```

Ce que votre shell repond

echo '\$USER' affiche **\$USER** litteralement. Rien n'est interprete entre simples quotes : ni le dollar, ni les pipes, ni les chevrons.

Ou c'est gere, auteur : Axel

src/expander/exp_var_replace.c : la fonction **update_state** rejoue le meme automate que le lexer, et l'expansion n'a lieu que si **state != STATE_SQUOTE**.

Ce que tu reponds

Le meme automate tourne deux fois, dans le lexer puis dans l'expander. C'est volontaire : le lexer decoupe sans comprendre, l'expander comprend sans redécouper.

10 a 12. env, export, unset

```
minishell$ env
minishell$ export TEST=42
minishell$ env | grep TEST
minishell$ export TEST=43
minishell$ unset TEST
minishell$ env | grep TEST # plus rien
minishell$ export 1BAD=x # nom invalide
```

Ce que votre shell répond

L'environnement est copie au démarrage dans une liste chaînée. **export** crée ou remplace, **unset** retire, **env** affiche. Un identifiant invalide est refusé avec un message.

Où c'est géré, auteur : Davi

src/env/env_init.c copie **envp** en liste chaînée de **t_env** (cle, valeur, drapeau exporté). **src/env/env_to_array.c** refabrique le tableau au moment du **execve**. **src/builtins/bi_export.c**, **bi_unset.c**, **bi_env.c**.

Le piège

La liste chaînée est copiée, jamais pointée sur **envp** : le tableau d'origine appartient au système et ne doit pas être modifié.

Ce que tu réponds

Le drapeau exporté distingue une variable créée par export d'une variable interne. env n'affiche que celles qui sont exportées, comme bash.

13 a 15. cd, pwd, chemins relatifs

```
minishell$ cd /tmp ; pwd
minishell$ cd .. ; pwd
minishell$ cd . ; pwd
minishell$ cd - ; pwd
minishell$ cd ../../.. ; /bin/ls
minishell$ cd /nexistepas ; echo $?
```

Ce que votre shell répond

cd sans argument va dans HOME, **cd -** revient et affiche le chemin, **PWD** et **OLDPWD** sont mis à jour. Une cible invalide affiche l'erreur système et rend 1.

Où c'est géré, auteur : Davi (cd) : Axel (pwd)

src/builtins/bi_cd.c : **get_target** choisit la cible, **chdir**, puis **update_pwd** récupère le chemin réel avec **getcwd** et met à jour les deux variables. **src/builtins/bi_pwd.c** utilise **getcwd(NULL, 0)** et libère le résultat.

Le piège

getcwd rend le chemin **physique** : après un **cd** à travers un lien symbolique, **pwd** affiche la cible réelle là où bash garde le chemin logique. C'est un écart assumé, le sujet n'exige pas le chemin logique.

Ce que tu réponds

*On appelle **getcwd** après chaque **chdir** plutôt que de bricoler la chaîne, parce que c'est le noyau qui a la vérité sur où on se trouve.*

16. Environment path

```
minishell$ ls
minishell$ unset PATH ; ls ; echo $? # 127
minishell$ export PATH=/dir1:/dir2 ; macommande
```

Ce que votre shell répond

Une commande sans slash est cherchée dans les répertoires du PATH, de gauche à droite. Sans PATH, on rend 127 avec *command not found*.

Ou c'est gere, auteur : Davi (ecriture initiale) : Axel (correction du 23/09)

src/executor/exec_path.c **resolve_path** distingue deux cas : un nom contenant un slash est teste directement, sinon **resolve_in_path** parcourt les repertoires.

Le piege

C'est ici qu'on a trouve un bug le 23 septembre. La recherche s'arretait au premier fichier du bon nom, meme non executable. Si un fichier non executable ou un repertoire du meme nom se trouvait dans un repertoire a gauche, il masquait le vrai binaire. Corrige : on continue jusqu'a trouver un executable.

Ce que tu reponds

Si aucun candidat n'est executable, on rend Permission denied (126) si on en a croise un, sinon command not found (127). C'est exactement ce que fait bash, et c'est verrouille par test_robustness.

17. Redirection

```
minishell$ echo salut > f ; cat f
minishell$ echo encore >> f ; cat f
minishell$ wc -l < f
minishell$ cat << EOF
minishell$ echo a > f1 > f2 > f3
minishell$ cat < fichierabsent ; echo $?
```

Ce que votre shell repond

Les quatre formes fonctionnent. Avec plusieurs redirections de sortie, toutes les cibles sont ouvertes et seule la derniere recoit les donnees, comme bash. Un fichier d'entree absent affiche l'erreur systeme et rend 1.

Ou c'est gere, auteur : Davi

src/redirections/redir_files.c : **open_redir** choisit les bons drapeaux (**O_TRUNC** pour >, **O_APPEND** pour >>), puis **apply_redirs** fait le **dup2** et referme tout de suite le descripteur d'origine.

Le piege

Le **close(fd)** juste apres le **dup2** est obligatoire. Sans lui, chaque redirection laisse un descripteur ouvert et on finit par epuiser la table de descripteurs du process.

Ce que tu reponds

Pour un builtin seul, on sauvegarde stdin et stdout avec dup avant d'appliquer les redirections, et on les restaure apres. Sinon un echo > f casserait la sortie du shell pour toujours.

18. Pipes

```
minishell$ cat Makefile | grep SRC | wc -l
minishell$ ls fichierabsent | grep bla | more
minishell$ cat | cat | ls
minishell$ ls | wc -l > out ; cat out
minishell$ cd /tmp | echo hi ; pwd # le cwd ne change pas
```

Ce que votre shell repond

Les pipelines de longueur quelconque fonctionnent. **\$?** vaut le code de la **derniere** commande. Un builtin dans un pipe s'execute dans un fils, donc **cd** et **export** n'ont aucun effet sur le shell pere, exactement comme bash.

Ou c'est gere, auteur : Davi

src/executor/exec_pipeline.c : pour chaque etage, on cree le tube **avant** le fork, le fils branche ses descripteurs avec **child_dup_pipes**, le pere referme son extremite immediatement. Tous les forks sont faits **avant** le premier wait.

Le piege

Forker tous les etages avant d'attendre est obligatoire. Si on attendait le premier fils avant de forker le second, un pipeline dont la premiere commande produit beaucoup de donnees remplirait le tube et se bloquerait : personne ne lit a l'autre bout.

Ce que tu reponds

`wait_all` garde le code du dernier fils attendu. C'est la semantique de `bash` sans `pipefail`.

19. Go Crazy and history

```
minishell$ du texte ctrl-C puis Entree
minishell$ fleche haut, fleche bas
minishell$ dsbksdgbksdghsd
minishell$ cat | cat | ls
minishell$ une commande avec 10 000 arguments
```

Ce que votre shell repond

L'historique fonctionne via **add_history**. Une commande inconnue affiche une erreur et rend 127 sans planter. **cat | cat | ls** se comporte normalement. Les lignes tres longues et les milliers d'arguments sont supportes depuis la correction du 23 septembre.

Ou c'est gere, auteur : Davi (main) : Axel (buffer dynamique)

src/main.c pour **add_history**, **src/lexer/lex_tokenize_utils.c** pour le buffer dynamique.

Le piege

C'est ici que la campagne adversariale a trouve le segfault : le buffer du lexer etait fige a 4096 octets sur la pile, et tout mot plus long ecrivait a cote. La section dit explicitement *try to execute a long command with a ton of arguments*. Voir la partie 3.

Ce que tu reponds

On ne stocke une ligne dans l'historique que si elle n'est pas vide, sinon on pollue l'historique avec des lignes blanches.

20 et 21. Environment variables et la section Surprise

```
minishell$ echo $USER
minishell$ echo "$USER"
minishell$ echo '$USER'
minishell$ echo "'$USER'"
minishell$ echo '"$USER"'
minishell$ echo $INEXISTANT
minishell$ echo a"b"c'd'$USER
```

Ce que votre shell repond

Ligne par ligne : la valeur, la valeur, le texte litteral, la valeur entre simples quotes, le texte **"\$USER"** litteral, une ligne vide, puis la concatenation **abcd** suivie de la valeur.

Ou c'est gere, auteur : Axel

src/expand/expand_run.c orchestre : d'abord l'**expansion** sur tous les mots, ensuite seulement le **retrait des quotes** via **exp_strip_tokens**. L'ordre est obligatoire.

Le piege

La section Surprise existe uniquement pour verifier cet ordre. Si vous retirez les quotes avant d'expanser, **echo '\$USER'** affiche la valeur au lieu du texte, et vous perdez la section.

Ce que tu reponds

Expansion d'abord, retrait des quotes ensuite. C'est la seule chose a retenir de tout l'expander, et c'est pour ca que exp_run tient en trois lignes.

Partie 2

Le code, module par module

Cette partie se lit **en croise**. Axel connaît le lexer, le parser et l'expander : il doit réviser l'exécuteur, les heredocs et les signaux. Davi connaît l'exécuteur : il doit réviser le lexer et l'expander. Le correcteur choisit qui il interroge, et il choisit souvent celui qui n'a pas écrit le fichier qu'il pointe.

Le chemin complet d'une ligne

```
l'utilisateur tape une ligne
|
v
+-----+
| readline() | <-- historique, fleches, ctrl-C
+-----+
| "echo \"$USER\" | wc -c > out"
v
+-----+
| syntax_check_line | quote non fermee ? -> erreur, $? = 2
+-----+
|
v
+-----+
| LEXER | automate 3 etats. Les quotes sont
| lex_tokenize / lex_quotes | CONSERVEES dans le token.
+-----+
| [WORD echo] [WORD "$USER"] [PIPE] [WORD wc] ...
v
+-----+
| syntax_check_tokens | | en bord, > sans cible -> $? = 2
+-----+
|
v
+-----+
| EXPANDER | 1. expansion de $VAR et $?
| exp_run = expand + strip | 2. PUIS retrait des quotes
+-----+ l'ordre est obligatoire
| [WORD echo] [WORD alvrd] [PIPE] [WORD wc] ...
v
+-----+
| PARSER | liste chaine de t_cmd,
| parse_input / parse_word | redirections accrochees
+-----+
| cmd1(echo alvrd) -> cmd2(wc -c, > out)
v
+-----+
| collect_heredocs | lit les << AVANT d'executer
+-----+
|
v
+-----+
| EXECUTOR | fork, pipe, dup2, execve, wait
| un builtin seul reste | sinon cd et export seraient
| dans le pere | perdus a chaque commande
+-----+
|
v
shell->last_exit mis a jour
|
v
free des tokens, de l'AST et de la ligne
|
v
retour a readline()
```

Le lexer, auteur : Axel

Son travail : transformer une chaine en liste de tokens. Il ne comprend rien a ce qu'il decoupe. Il sait juste reconnaitre les separateurs, les operateurs, et se souvenir s'il est a l'interieur d'une paire de quotes.

L'automate a trois etats

```
28 void lex_handle_quote_char(t_lex *lex, char c)
29 {
30     if (!lex_in_quote(lex))
31     {
32         if (c == '"')
33             lex->state = STATE_DQUOTE;
34         else
35             lex->state = STATE_SQUOTE;
36         lex->token_active = 1;
37         lex->has_quotes = 1;
38         append_char(lex, c);
39         return ;
40     }
41     if ((lex->state == STATE_SQUOTE && c == '\\')
42         || (lex->state == STATE_DQUOTE && c == '"'))
43     {
44         lex->state = STATE_NORMAL;
45         append_char(lex, c);
46         return ;
47     }
48     lex->token_active = 1;
49     append_char(lex, c);
50 }
```

Ce que ca fait

Trois situations. Hors quote, un guillemet ouvre l'etat correspondant. Dans une quote, **seul** le meme caractere la referme. Tout le reste est accumule tel quel.

Pourquoi comme ca

Une simple quote a l'interieur de guillemets doit rester un caractere ordinaire. Si on testait juste *est-ce un caractere de quote*, **echo "il a dit 'salut'"** ouvrirait puis fermerait des etats au hasard. C'est pour ca que la fermeture compare au caractere qui a ouvert.

Le piege

append_char(lex, c) ajoute la quote **dans** le token. On la garde volontairement : l'expandre en aura besoin pour savoir si un dollar doit etre expanse.

Le buffer dynamique (correction du 23 septembre)

```
15 static int grow_buffer(t_lex *lex)
16 {
17     char *bigger;
18
19     bigger = malloc(lex->cap * 2);
20     if (!bigger)
21     {
22         free(lex->buffer);
23         lex->buffer = NULL;
24         return (0);
25     }
26     ft_memcpy(bigger, lex->buffer, lex->len + 1);
27     free(lex->buffer);
28     lex->buffer = bigger;
29     lex->cap = lex->cap * 2;
30     return (1);
31 }
32
33 void append_char(t_lex *lex, char c)
```

```

34 {
35 if (!lex->buffer)
36 return ;
37 if (lex->len + 2 > lex->cap && !grow_buffer(lex))
38 return ;
39 lex->buffer[lex->len] = c;
40 lex->len++;
41 lex->buffer[lex->len] = '\0';
42 }

```

Ce que ca fait

Le tampon du token grandit par doublement, a partir de 64 octets. On garde la longueur courante dans **lex->len** au lieu de la recalculer.

Pourquoi comme ca

La version d'origine avait **char buffer[4096]** dans la structure et ecrivait sans verifier la borne : tout mot de 4096 caracteres faisait segfauter le shell. Elle appelait aussi **ft_strlen** a chaque caractere, ce qui rendait le decoupage quadratique. Les deux problemes disparaissent avec un compteur et une capacite.

Le piege

Si le **malloc** echoue, on met **buffer** a NULL plutot que de tronquer en silence. **lex_tokenize** le detecte et abandonne la ligne. Tronquer serait pire qu'echouer : le shell executerait une commande que l'utilisateur n'a pas tapee.

L'expander, auteur : Axel

C'est le module le plus interroge en soutenance, parce que c'est celui ou l'ordre des operations est une decision et non une evidence.

Le point d'entree, en trois lignes

```

1 int exp_run(t_shell *shell)
2 {
3     if (expand_tokens(shell))
4         return (1);
5     return (exp_strip_tokens(shell->tokens));
6 }

```

Ce que ca fait

D'abord on remplace les variables dans tous les tokens, ensuite seulement on retire les quotes delimitantes.

Pourquoi comme ca

L'inverse casse tout. Avec **echo '\$USER'** : si on retire les quotes d'abord, le token devient **\$USER** et l'expansion le remplace par la valeur. Le sujet et la section Surprise de la grille exigent le texte litteral. Cet ordre est la raison d'etre du module.

L'expansion, quote-aware

```

89 char *exp_var_replace(t_shell *shell, char *str, int respect_quotes)
90 {
91     char *res;
92     t_lex_state state;
93     int i;
94     int n;
95
96     res = ft_strdup("");
97     state = STATE_NORMAL;
98     i = 0;
99     while (res && str[i])
100     {
101         if (respect_quotes)

```

```

102 update_state(&state, str[i]);
103 if (str[i] == '$' && state != STATE_SQUOTE)
104     i += expand_one(shell, str + i, &res);
105 else
106     {
107         n = plain_len(str + i, state, respect_quotes);
108         append(&res, str + i, n);
109         i += n;
110     }
111 }
112 return (res);
113 }

```

Ce que ca fait

On reparcourt la chaine en jouant l'automate des quotes. Un dollar declenche une expansion sauf si on est entre simples quotes. Sinon, on copie d'un bloc tous les caracteres litteraux consecutifs.

Pourquoi comme ca

Le parametre **respect_quotes** vaut 0 pour un corps de heredoc : la, les quotes sont du texte, pas de la structure, donc une apostrophe ne doit pas couper l'expansion. C'est le meme code pour les deux usages, avec un seul drapeau.

Le piege

plain_len a ete ajoute le 23 septembre. Avant, on appelait **ft_strjoin** une fois par caractere : chaque appel recopiait toute la chaine accumulee, donc le cout etait en n au carre. Une ligne de 100 000 caracteres mettait 28 secondes. Elle met moins d'une seconde.

Le piege du malloc de taille zero

```

22 static void append(char **res, char *add, int len)
23 {
24     char *piece;
25     char *joined;
26
27     if (!*res || len == 0)
28         return ;
29     piece = ft_substr(add, 0, len);

```

Ce que ca fait

Si la longueur a coller est nulle, on sort tout de suite.

Pourquoi comme ca

Notre **ft_substr** renvoie NULL pour une longueur de zero. Sans ce test, on prendrait ce NULL pour un echec de malloc, on mettrait **res** a NULL, et **echo \$INEXISTANT** a n'afficherait rien du tout en rendant 1. Le bug a coute une demi-journee : les tests unitaires le montraient, pas les tests d'integration.

Le piege

Si un correcteur demande un exemple de bug difficile, c'est celui-la qu'il faut raconter. Il montre qu'on connait notre libft et qu'on sait lire une valeur de retour ambigue.

L'executeur et le pipeline, auteur : Davi

Axel, c'est cette page qu'il faut savoir redire sans la lire.

Un etage de pipeline

```

15 static pid_t spawn_stage(t_shell *shell, t_cmd *cmd, int prev_read,
16 int *next_read)
17 {
18     int p[2];

```

```

19  pid_t pid;
20
21  *next_read = -1;
22  if (cmd->next)
23  pipe(p);
24  pid = fork();
25  if (pid == 0)
26  {
27  child_dup_pipes(prev_read, p, cmd->next != NULL);
28  pipe_child_run(shell, cmd);
29  }
30  if (prev_read != -1)
31  close(prev_read);
32  if (cmd->next)
33  {
34  close(p[1]);
35  *next_read = p[0];
36  }
37  return (pid);
38  }

```

Ce que ca fait

Le tube est cree **avant** le fork, sinon le fils ne le verrait pas. Le fils branche ses descripteurs puis part executer. Le pere referme immediatement ce qui ne le concerne plus et garde l'extremite de lecture pour l'etage suivant.

Pourquoi comme ca

Les **close** du pere sont la partie qu'on oublie et qui bloque tout. Tant qu'un seul process garde l'extremite d'ecriture ouverte, le lecteur a l'autre bout ne recoit jamais sa fin de fichier et reste bloque dans **read** pour toujours.

Le piege

Question classique : *pourquoi ne pas attendre chaque fils juste apres l'avoir cree ?* Parce que le tube a une capacite finie, de l'ordre de 64 Ko. Si la premiere commande ecrit plus que ca et que personne ne lit, elle se bloque, et vous attendez un fils qui n'avancera jamais. Il faut forker tout le pipeline puis attendre.

Recuperer le code de sortie

```

15  int exec_wait(pid_t pid)
16  {
17  int status;
18
19  if (waitpid(pid, &status, 0) < 0)
20  return (1);
21  if (WIFEXITED(status))
22  return (WEXITSTATUS(status));
23  if (WIFSIGNALED(status))
24  return (128 + WTERMSIG(status));
25  return (1);
26  }

```

Ce que ca fait

waitpid rend un entier encode. **WIFEXITED** dit si le fils est sorti normalement, **WEXITSTATUS** en extrait le code. **WIFSIGNALED** dit s'il a ete tue, et on rend alors **128 + numero du signal**.

Pourquoi comme ca

Le **status** de **waitpid** n'est pas un code de sortie : c'est un entier ou le code, le signal et l'information de core dump sont ranges dans des bits differents. Lire **status** directement donnerait des valeurs absurdes comme 256 pour un exit 1.

Le piege

Le 130 de ctrl-C, c'est 128 + 2. Le 131 de ctrl-backslash, c'est 128 + 3. Si on vous demande d'ou sort le 130, c'est la reponse.

Pourquoi un builtin seul reste dans le pere

```
41 void exec_run(t_shell *shell)
42 {
43     t_cmd *cmd;
44
45     cmd = shell->cmds;
46     if (!cmd)
47         return ;
48     if (cmd->next)
49     {
50         exec_pipeline(shell);
51         return ;
52     }
53     if (cmd->argv && cmd->argv[0] && !is_builtin(cmd->argv[0]))
54         exec_single(shell, cmd);
55     else
56         shell->last_exit = run_in_parent(shell, cmd);
57 }
```

Ce que ca fait

Trois chemins : un pipeline part dans **exec_pipeline**, une commande externe seule dans **exec_single**, un builtin seul reste dans le process courant.

Pourquoi comme ca

Un **cd** execute dans un fils changerait le repertoire du fils, qui meurt aussitot. Le pere n'aurait pas bouge. Meme chose pour **export** et **unset** : leur effet doit survivre a la commande. En revanche, dans un pipeline, bash lui-meme les met dans un fils, donc **cd /tmp | echo hi** ne change rien au repertoire. On reproduit ce comportement.

Le piege

run_in_parent sauvegarde stdin et stdout avec **dup** avant d'appliquer les redirections et les restaure ensuite. Sans ca, un **echo salut > f** laisserait la sortie du shell branchee sur le fichier pour le reste de la session.

Les heredocs et les signaux, auteur : Davi

Annuler un heredoc avec ctrl-C

```
15 static void sig_handler_heredoc(int signum)
16 {
17     (void)signum;
18     g_signal = SIGINT;
19     write(STDOUT_FILENO, "\n", 1);
20 }
21
22 void setup_heredoc_signals(void)
23 {
24     struct sigaction sa;
25
26     sa.sa_handler = sig_handler_heredoc;
27     sigemptyset(&sa.sa_mask);
28     sa.sa_flags = 0;
29     sigaction(SIGINT, &sa, NULL);
30     sa.sa_handler = SIG_IGN;
31     sigaction(SIGQUIT, &sa, NULL);
32 }
```

Ce que ca fait

Pendant la lecture d'un heredoc, SIGINT note son passage dans la globale et affiche un retour a la ligne.

Pourquoi comme ca

Le detail qui fait tout, c'est **sa_flags = 0**. Le handler du prompt, lui, utilise **SA_RESTART**. Ici on ne le veut surtout pas : sans **SA_RESTART**, le **read** bloquant qui lit la ligne du heredoc est interrompu et renvoie **-1** avec **errno = EINTR**. C'est ce retour qui permet a **read_heredoc_body** de rendre la main et a **collect_one_heredoc** d'annuler.

Le piege

Avec **SA_RESTART**, le read serait automatiquement relance apres le handler et le ctrl-C n'aurait aucun effet visible. C'est exactement le genre de detail que le correcteur teste sans le formuler.

Vider le buffer de readline apres ctrl-C

```
17 static void sig_handler_prompt(int signum)
18 {
19     (void)signum;
20     g_signal = SIGINT;
21     write(STDOUT_FILENO, "\n", 1);
22     rl_on_new_line();
23     rl_replace_line("", 0);
24     rl_redisplay();
25 }
```

Ce que ca fait

Trois appels de readline, dans cet ordre : on declare qu'on est sur une nouvelle ligne, on remplace le contenu de la ligne en cours par une chaine vide, on redessine le prompt.

Pourquoi comme ca

La grille exige que le buffer soit vide apres ctrl-C : *press Enter to make sure nothing from the previous line is executed*. Sans **rl_replace_line**, readline garderait ce que l'utilisateur avait tape et l'enverrait a l'execution au prochain Entree. C'est une commande que personne n'a validee qui partirait.

Le piege

rl_replace_line n'existe pas dans la libedit d'Apple. C'est pour ca que le Makefile pointe vers la readline de Homebrew sur macOS. Sans ce bloc, le projet ne compile pas sur Mac, donc pas sur la machine de l'eval.

Partie 3

Qui a écrit quoi

Mesure par **git blame** sur les 48 fichiers du depot, le 23 septembre. La colonne *auteur* est celui qui detient la majorite des lignes. Servez-vous en pour la demo croisee : **chacun revise la colonne de l'autre**.

Zone	Auteur principal	Contenu
src/lexer/ (6 fichiers)	Axel	Automate de quotes, decoupage, operateurs, liste de tokens
src/parser/ cmd_list, parse_*, syntax_check	Axel	Tokens vers liste de t_cmd, redirections, validation
src/parser/parse_heredoc*.c	Davi	Lecture des heredocs, fichiers temporaires, expansion du corps
src/expander/ (5 fichiers)	Axel	\$VAR, \$?, retrait des quotes, ordre des operations
src/executor/exec_run.c	Axel	Dispatch : pipeline, commande externe, ou builtin dans le pere
src/executor/ (7 autres)	Davi	fork, pipe, dup2, resolution PATH, wait, codes de sortie
src/redirections/redir_files.c	Davi	open avec les bons drapeaux, dup2, fermeture des fd
src/signals/ (2 fichiers)	Davi	Handlers prompt, execution, heredoc
src/env/ (3 fichiers)	Davi	Copie de envp en liste chainee, acces, retour en tableau
src/builtins/ cd, env, export, unset	Davi	Quatre builtins qui touchent a l'environnement
src/builtins/ echo, pwd, exit	Axel	Trois builtins sans effet de bord sur l'environnement
src/utls/ ut_init_data.c, ut_str.c	Axel	run_line (le chainage complet), helpers de chaines
src/utls/ ut_cleanup.c, ut_error.c	Davi	Liberation globale, messages d'erreur sur fd 2
src/main.c	Davi	Boucle REPL, readline, historique, sortie sur ctrl-D
includes/minishell.h	Davi (122 l.) et Axel (90 l.)	Structures et prototypes, ecrit a deux
tests/ et .github/workflows/	Axel	16 suites, 414 assertions, CI build + norme + tests + valgrind

Le plan de revision croisee, une heure a deux

Axel explique a Davi : l'automate a trois etats du lexer, pourquoi les quotes sont conservees jusqu'a l'expander, l'ordre expansion puis strip dans exp_run, et le bug de ft_substr a longueur zero.

Davi explique a Axel : pourquoi le pipe est cree avant le fork, pourquoi le pere ferme ses extremités tout de suite, pourquoi tous les forks precedent le premier wait, le role de WIFEXITED contre WIFSIGNALED, et pourquoi le handler de heredoc n'a pas SA_RESTART.

Ensuite vous **inversez** : chacun redit la partie de l'autre, sans le code sous les yeux. Ce qui ne ressort pas est ce qu'il faut relire.

La campagne adversariale du 23 septembre

Huit agents ont attaque le shell en parallele, un par axe de la grille, en comparant systematiquement a bash 5.2 et en pilotant un vrai pseudo-terminal pour les signaux. Environ 900 cas testes. Chaque trouvaille est ensuite passee devant un verificateur dont la consigne etait de la **refuter**. Voila ce qui en est sorti.

Corrige le jour meme

Ce qui n'allait pas	Effet	Correctif
Buffer du lexer fige a 4096 octets sur la pile, ecriture sans borne	Segfault des qu'un mot depasse 4095 caracteres. Note eliminatoire.	Buffer dynamique par doublement, longueur suivie, echec de malloc gere
exp_var_replace collait un caractere a la fois avec ft_strjoin	Quadratique. 28 s pour une ligne de 100 000 caracteres, plus de 40 s a 200 000.	plain_len regroupe les caracteres litteraux : moins d'une seconde a 1 000 000
resolve_in_path s'arretait au premier candidat du bon nom	Un fichier non executable ou un repertoire homonyme a gauche masquait le vrai binaire. La grille teste l'ordre du PATH.	On continue jusqu'a trouver un executable, 126 si on en a croise un sans droit, 127 sinon

La lecon a retenir de ces trois bugs

Aucun des trois n'etait attrape par les 399 assertions existantes, et pourtant les suites etaient vertes. Pire : un test unitaire **encodait** le comportement fautif du PATH, donc il defendait le bug.

Si un correcteur vous demande comment vous savez que votre code est juste, la bonne reponse n'est pas *on a des tests*. C'est *on a compare a bash, cas par cas, et on a ajoute un test quand on a trouve un ecart*. Un test vert prouve que le code fait ce que le test attend, pas qu'il est correct.

Les ecarts avec bash que l'on assume

Le sujet est explicite : *You should limit yourself to the subject description. Anything that is not asked is not required*. Les points ci-dessous sont de vraies differences avec bash, mais hors du perimetre obligatoire. Si le correcteur les souleve, la reponse est la meme a chaque fois : **ce n'est pas demande**, et on sait pourquoi on ne l'a pas fait.

Ce que bash fait et nous pas	Ce que tu reponds
\${VAR} avec accolades	Le sujet demande l'expansion de \$VAR et de \$? . Les accolades sont une syntaxe de substitution de parametre, avec les valeurs par default et les sous-chaines. Ce n'est pas dans le mandatory.
Word splitting apres expansion : une variable contenant des espaces donne plusieurs mots	Le sujet ne le demande pas et la plupart des minishell ne le font pas. Ca impliquerait de redécouper apres expansion, donc de relancer le lexer sur le resultat. On a choisi de ne pas le faire pour garder une pipeline a sens unique.
\$\$, \$0, \$@, \$#, \$*	Parametres speciaux et positionnels. Le sujet n'exige que \$VAR et \$?.
<<- (heredoc qui mange les tabulations)	Le sujet demande <<, pas <<-.
~ et ~user	L'expansion du tilde n'est pas au programme du mandatory.
Prompt de continuation sur une quote non fermee	Bash ouvre un prompt secondaire. Le sujet ne demande pas le multi-ligne, on rend une erreur de syntaxe avec \$? a 2.
cd garde le chemin logique a travers un lien symbolique	On appelle getcwd, qui rend le chemin physique. C'est la verite du noyau. Le sujet ne tranche pas.
Un fichier sans shebang est relance via /bin/sh	Bash rattrape ENOEXEC. Nous rendons une erreur. Cas limite hors mandatory.

La phrase qui clot le sujet

C'est une vraie difference avec bash, on l'a mesuree. Le sujet dit de s'en tenir a ce qui est demande, et ce point n'y est pas. On sait comment on l'implementerait si c'etait au programme.

Puis vous expliquez comment. C'est cette deuxieme phrase qui fait la difference entre une lacune et un choix.

Les questions pieges, et vos reponses

Combien de variables globales, et pourquoi ?

Une seule : **int g_signal**, dans `src/signals/signals.c`. Un gestionnaire de signal ne recoit qu'un numero et ne rend rien : il ne peut communiquer avec le reste du programme que par une variable de portee globale. On y met uniquement le numero du signal reçu. Tout le reste de l'etat du shell, l'environnement, le dernier code de sortie, les tokens, vit dans la structure **t_shell** qu'on passe en parametre.

L'appui concret

Le sujet interdit qu'une globale donne acces a autre chose que le numero du signal. La notre est un int, pas un pointeur, donc elle ne peut rien donner d'autre par construction.

Pourquoi expander avant de retirer les quotes, et pas l'inverse ?

Parce que les quotes portent l'information qui decide si l'expansion doit avoir lieu. Une fois retirees, cette information est perdue. Avec **echo '\$USER'**, si on strip d'abord, le token devient `$USER` et l'expansion le remplace par la valeur, alors qu'il faut afficher le texte. C'est exactement ce que teste la section Surprise de la grille.

L'appui concret

`exp_run` tient en trois lignes justement pour que cet ordre soit lisible d'un coup d'oeil.

Pourquoi forker tous les etages du pipeline avant le premier wait ?

Parce qu'un tube a une capacite finie, de l'ordre de 64 Ko. Si on attend le premier fils avant de creer le second, et que le premier ecrit plus que la capacite du tube, il se bloque sur son write : personne ne lit a l'autre bout, puisque le lecteur n'existe pas encore. Le shell attend un fils qui n'avancera jamais. On cree donc tous les process, puis on les attend.

L'appui concret

C'est aussi pour ca que le pere ferme ses extremités de tube immediatement : tant qu'un process garde l'écriture ouverte, le lecteur ne recoit jamais sa fin de fichier.

Pourquoi un builtin seul ne part-il pas dans un fils ?

Parce que son effet doit survivre a la commande. Un **cd** dans un fils change le repertoire du fils, qui meurt aussitot : le shell n'a pas bouge. Meme raisonnement pour **export** et **unset**. En revanche, dans un pipeline, **bash** lui-meme les met dans un fils, donc **cd /tmp | echo hi** ne change rien. On reproduit ce comportement.

L'appui concret

`run_in_parent` sauvegarde `stdin` et `stdout` avec `dup` avant d'appliquer les redirections, et les restaure apres, sinon un `echo > f` redirigerait la sortie du shell pour toute la session.

D'ou sort le 130 de ctrl-C ?

De **128 + WTERMSIG(status)**. `SIGINT` vaut 2, donc 130. `SIGQUIT` vaut 3, donc 131. C'est la convention POSIX pour distinguer un process tue par un signal d'un process sorti normalement avec un petit code. On lit le status de `waitpid` avec `WIFEXITED` et `WIFSIGNALED`, jamais directement : c'est un entier ou le code, le signal et le core dump occupent des bits differents.

L'appui concret

Au prompt, quand `ctrl-C` n'a tue aucun fils, c'est `check_prompt_signal` dans `main` qui pose 130 a partir de la globale.

Et les fuites memoire ?

Zero **definitely lost** et zero **indirectly lost**, verifie sous valgrind sans aucun fichier de suppression, sur 31 scenarios. Les blocs **still reachable** qui restent viennent tous de libreadline et libtinfo : ce sont les keymaps et le terminfo alloues une fois a l'initialisation et gardes en variables globales de la bibliotheque. Le sujet, page 9, dit que les leaks de readline ne sont pas de notre responsabilite.

L'appui concret

Un job de la CI relance valgrind a chaque push. Si l'un de nous introduit une fuite, le build passe au rouge avant le merge.

Comment gerez-vous l'echec d'un malloc ?

Chaque fonction qui alloue rend NULL en cas d'echec, et l'appelant remonte. Dans l'expand, **exp_var_value** rend **toujours** de la memoire allouee, meme pour une variable inexistante ou vide : comme ca l'appelant fait un free inconditionnel, sans avoir a se demander qui possede quoi. Dans le lexer, un echec de croissance du buffer met le pointeur a NULL, ce que lex_tokenize detecte pour abandonner la ligne au lieu de la tronquer en silence.

L'appui concret

Tronquer serait pire qu'echouer : le shell executerait une commande que l'utilisateur n'a pas tapee.

Pourquoi une liste chaine pour l'environnement plutot qu'un tableau ?

Parce que unset doit retirer un element au milieu et export doit en ajouter, sans realouer ni decaler tout le reste. Le tableau n'est refabrique qu'au moment ou on en a vraiment besoin, c'est-a-dire juste avant l'execve, par env_to_array, puis libere.

L'appui concret

On copie envp au demarrage, on ne pointe jamais dessus : ce tableau appartient au systeme.

Qu'est-ce qui vous a pris le plus de temps ?

Le bug de ft_substr avec une longueur de zero. Notre libft rend NULL dans ce cas, ce qu'on prenait pour un echec d'allocation, donc **echo \$INEXISTANT a** n'affichait rien et rendait 1. Les tests unitaires le montraient, les tests d'integration non, parce qu'ils ne regardaient que la sortie. Depuis, la regle est de comparer a bash plutot que de comparer a ce qu'on attend.

L'appui concret

C'est la reponse a donner si on vous demande un exemple de bug difficile : elle montre qu'on connait notre libft et qu'on sait lire une valeur de retour ambigue.

La checklist du jour J

Avant que le correcteur clone

1. **git clone** dans un dossier vide, puis **make** : c'est ce que le correcteur va faire, faites-le une dernière fois vous-mêmes. Un fichier oublié dans le `.gitignore` et c'est fini.
2. Vérifiez qu'aucun alias n'est en place dans votre shell. La grille demande explicitement au correcteur de chercher des alias malicieux : si vous en avez un innocent, expliquez-le avant qu'il le trouve.
3. **make -n | grep -- -Wall** pour montrer les flags sans avoir à ouvrir le Makefile.
4. **make** une seconde fois : *Nothing to be done*, pas de relink.
5. **norminette src/ includes/ libft/** : zéro erreur.
6. Les 16 suites de tests, 414 assertions, en une commande. Proposez-la, ne l'imposez pas : la grille demande de revoir ensemble tout script de test avant de s'en servir.

Pendant la soutenance

Ne touchez jamais au clavier du correcteur. S'il tape une commande qui vous surprend, laissez-la finir. Un shell qui plante parce que vous avez voulu l'empêcher de taper, c'est le même zéro.

Si ça plante, dites-le avant lui. Nommer soi-même le problème et expliquer pourquoi vaut toujours mieux que le laisser découvrir et vous voir chercher.

Ne dites jamais *je ne sais pas, c'est Davi qui l'a fait*. C'est la phrase qui coûte le plus cher en soutenance de binôme. Si vous ne savez pas, dites ce que vous savez du module et raisonnez à voix haute : le correcteur note la compréhension, pas la recitation.

Le sujet est votre argument. Pour tout ce qui n'est pas demandé, citez-le. C'est écrit noir sur blanc : *Anything that is not asked is not required*.

La timeline du projet

Du 26 mai au 23 septembre 2026, 96 commits, 2 personnes. Utile si le correcteur demande comment vous vous organisez.

Periode	Ce qui a été fait	Qui
26 mai	Organisation du dépôt, guide d'équipe, squelette de tests, README	Axel
4 juin	Fondations : structures, squelette de main, Makefile	Davi
15 juin au 28 juillet	Environnement : copie de envp, API get/set/find, retour en tableau, affichage	Davi
2 au 15 juillet	Lexer : découpage sur les blancs, détection des opérateurs	Axel
1er au 11 août	Lexer : automate de quotes à trois états	Axel
12 au 27 août	Messages d'erreur, libération mémoire, tests d'environnement	Davi
8 au 17 septembre	Parser : tokens vers <code>t_cmd</code> , redirections. Builtins echo, pwd, exit. Chainage REPL et validation syntaxique	Axel
11 au 18 septembre	Exécuteur : résolution du PATH, fork, redirections, pipeline, export, heredoc	Davi
21 septembre	Expander complet en quatre PR : longueur du nom, valeur, remplacement, tests de bout en bout. Mise en place de la CI	Axel
21 septembre	unset, tests de pipes et de redirections, correction de cd pour macOS, signaux	Davi
22 septembre	Fuites de descripteurs, bug de heredoc, tabulation comme séparateur	Davi
23 septembre	Correction du build macOS, job valgrind dans la CI, nettoyage des fichiers vides, README, campagne adversariale et ses trois correctifs	Axel

S'il ne fallait retenir que six phrases

1. Une seule globale, un int, parce qu'un handler de signal ne peut écrire nulle part ailleurs.
2. Les quotes survivent jusqu'à l'expander, parce qu'elles décident si un dollar s'expande.
3. On expande puis on strip, jamais l'inverse.
4. On forke tout le pipeline avant d'attendre, parce qu'un tube a une capacité finie.
5. Un builtin seul reste dans le père, parce que cd doit survivre à la commande.
6. 128 plus le numéro du signal, lu avec WIFSIGNALED, jamais le status brut.